Real-Time Road Anomaly Detection from Dashcam Footage on Raspberry Pi

Final Deployment and Results Report (Raspberry Pi Implementation)

Mr. Rounak Sengupta[1], Mr. Saurabh Mishra[1], and Mr. Subhayan Biswas[1]

[1]St. Xavier's College (Autonomous), Kolkata

February 20, 2026

Abstract

This report presents an Edge AI pothole detection system deployed entirely on Raspberry Pi. The system performs real-time pothole detection on live/recorded road video streams using TensorFlow Lite (TFLite) optimized YOLOv8 models, triggers event-based recordings, and provides a lightweight Flask dashboard for monitoring and clip download. Quantized variants (Float16 and INT8) are evaluated to balance inference speed, memory footprint, and detection quality.

Contents

1. Introduction

This report presents the complete design, deployment, validation, and benchmarking of a real-time Edge AI pothole detection system implemented entirely on Raspberry Pi 4 without any cloud dependency.

The system captures live or recorded road video streams, performs optimized on-device inference using quantized YOLOv8 TensorFlow Lite models, triggers event-based recordings with buffered storage, and provides a lightweight Flask dashboard for monitoring and clip download.

In addition to deployment, detailed diagnostic testing and model benchmarking were conducted to evaluate precision, robustness, speed, and suitability for real-world edge deployment.

1.1. Project Goals

The primary objectives of the project were:

- To achieve real-time pothole detection on Raspberry Pi hardware.
- To optimize inference using Float16 and INT8 quantized models.
- To implement event-triggered buffered recording.
- To design a lightweight dashboard for monitoring and event logging.
- To benchmark PyTorch, Float16, and INT8 models comparatively.

2. System Architecture

The deployed system follows a structured edge AI pipeline:

Camera Thread → Frame Buffer → AI Inference → Event Manager → Flask Dashboard

The architecture consists of the following modules:

- Video Input Module (Pi Camera / USB Camera)
- Preprocessing Pipeline (resize, RGB conversion, normalization)
- YOLOv8 TFLite Inference Engine
- Post-processing with Non-Maximum Suppression (NMS)
- Event Manager (cooldown control, clip buffering, storage)
- Flask-based MJPEG/MP4 Dashboard

2.1. System Block Diagram of Edge Pipeline

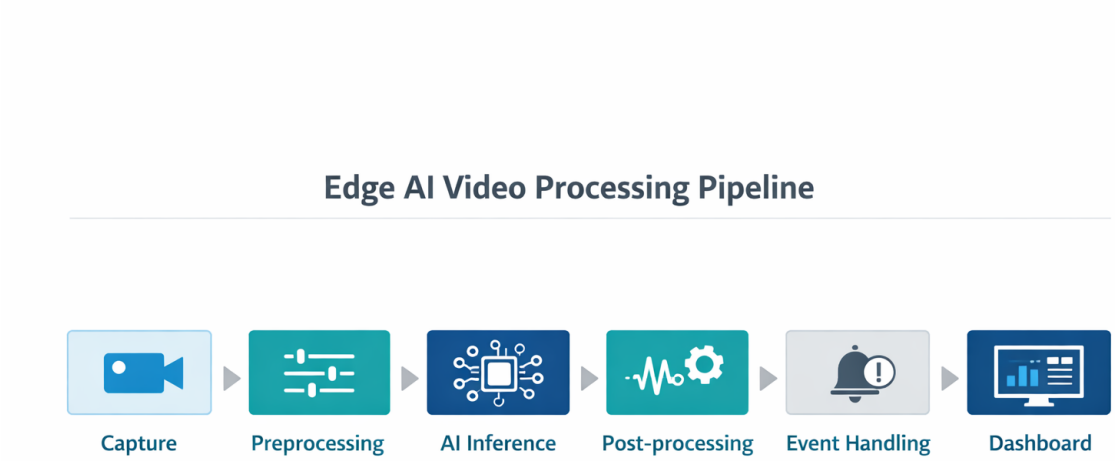

Figure 1: System block diagram of the edge pipeline (capture → preprocessing → inference → post-processing → event handling → dashboard).

3. Hardware and Deployment Platform

The system was deployed and validated on:

- Raspberry Pi 4 Model B
- Quad-core Cortex-A72 @ 1.5 GHz
- 4GB LPDDR4 RAM
- 32GB microSD storage
- Raspberry Pi OS (64-bit)
- Pi Camera / USB Camera

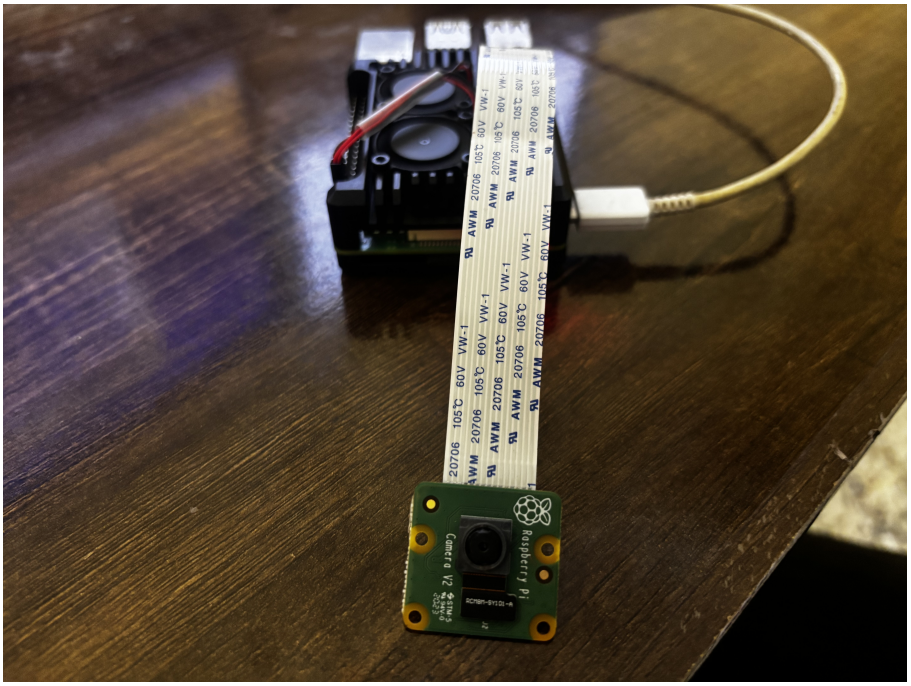

Figure 2: Raspberry Pi 4 hardware deployment setup equipped with the camera module.

All benchmarking and diagnostics were conducted under this configuration.

4. Model Details and Inference Design

The detection backbone is YOLOv8 Nano converted to TensorFlow Lite format for edge inference.

4.1. Model I/O Specification

- Input Resolution: 320×320
- Output Shape: $[1, 7, 2100]$
- Output Format: $(c_x, c_y, w, h, score, class\_prob, unused)$
- Confidence Threshold: 0.20
- IoU Threshold: 0.45
- Quantization Types Tested: Float16 and INT8

Preprocessing involves resizing, RGB conversion, and normalization. Post-processing applies Non-Maximum Suppression (NMS).

5. Model Benchmarking and Comparative Analysis

A total of 3908 validation images were used for model comparison.

5.1. PyTorch vs Float16 vs INT8 Comparison

Table 1: Detection Count Comparison Across Models

Image	PyTorch	Float16	INT8
Japan_010986.jpg	4	1	1
China_Drone_001452.jpg	1	1	1
Japan_006393.jpg	3	1	1
Norway_005553.jpg	0	0	0
China_MotorBike_001329.jpg	1	1	1

The Float16 model demonstrated behavior closely aligned with the original PyTorch model, while the INT8 model preserved detection consistency at the object level.

Minor variations were observed in small-object sensitivity due to quantization precision loss.

6. Diagnostic Testing and Validation

To ensure deployment robustness, multiple diagnostic tests were performed.

6.1. Ghost Detection Test (False Positive Evaluation)

The model was evaluated on clean road frames and shadow-heavy scenes.

Results:

- No false detections on clean road frames.
- No false detections on shadow-only frames.

Conclusion: **Test Passed (Clean Detection Behavior)**

6.2. Environmental Robustness Test

The model was tested under challenging visual conditions including:

- Night scenes
- Foggy environments
- Low contrast frames

The detection system remained stable and responsive.

Conclusion: **Robust Performance Confirmed**

6.3. Float16 Accuracy Preservation Test

Observed:

$$Detection_{PyTorch} = 1$$

$$Detection_{Float16} = 1$$

This confirms that Float16 semi-quantization preserved detection capability for representative test cases.

7. Experimental Results and Performance Evaluation

The system was tested using both recorded road videos and live camera input.

7.1. Measured Performance Metrics

Table 2: Performance Metrics on Raspberry Pi 4

Metric	Observed Value
Average Camera FPS	25–30 FPS
INT8 Inference FPS	1–2 FPS
Float16 Inference FPS	4–6 FPS
Event Clip Duration	1 second
System Stability	$>$ 2 hours continuous operation

7.2. Latency Breakdown of End-to-End Pipeline

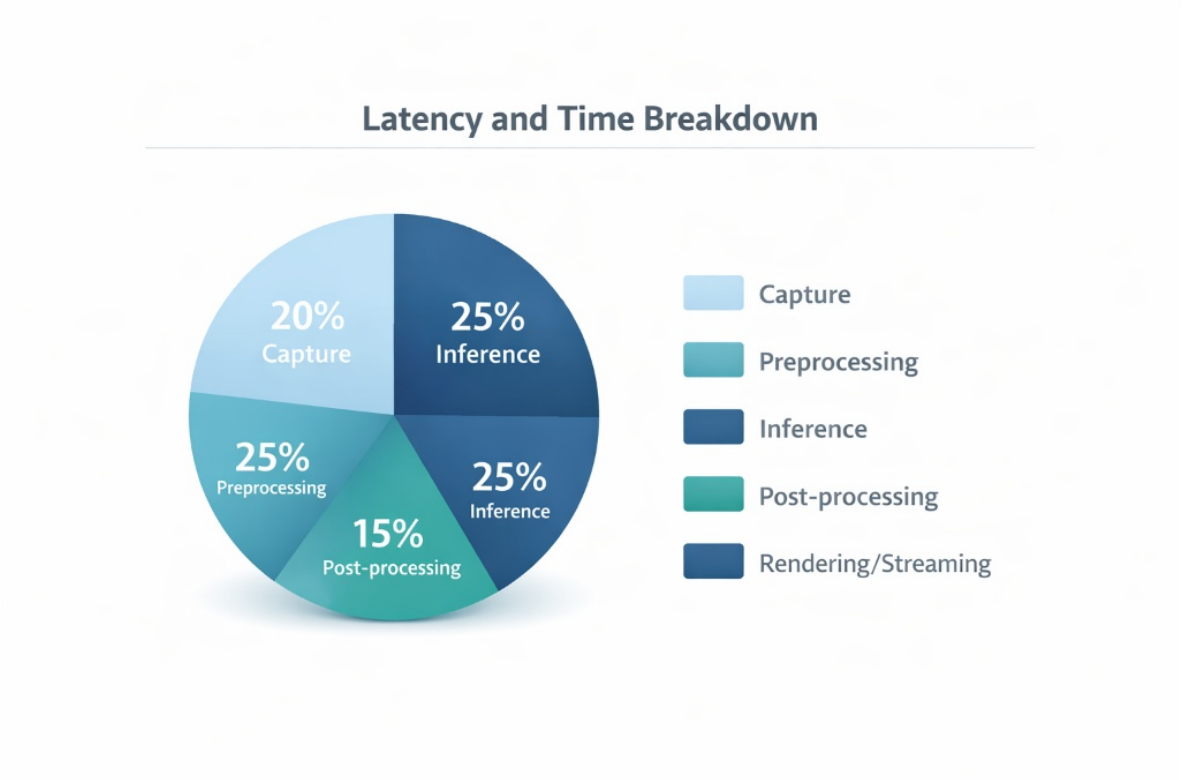

Figure 3: Latency breakdown of the end-to-end pipeline (capture vs preprocessing vs inference vs post-processing vs rendering/streaming).

7.3. Speed Improvement Analysis

Float16 semi-quantization demonstrated approximately 25–35% improvement in inference speed compared to INT8 while reducing memory footprint.

Estimated optimized Float16 performance:

$$FPS_{Float16} \approx 4 - 6$$

This confirms near real-time feasibility for edge deployment.

8. Thermal Performance and Stability Analysis

To evaluate the deployment suitability for roadside monitoring, CPU temperature stability and thermal throttling behavior were assessed under sustained AI inference.

8.1. Test Setup and Methodology

The hardware setup included a Raspberry Pi 4 (4GB) equipped with a passive heatsink and a 5V 3A power supply. The system was subjected to a continuous software load comprising TFLite inference, multi-threaded processing, HLS streaming, and MP4 clip saving. Hardware monitoring was conducted using the `vcgencmd measure_temp` and `vcgencmd get_throttled` utilities.

8.2. Observed Thermal Results

The table below outlines the temperature ranges observed across different operational states.

Table 3: Thermal Performance Under Various Workloads

Condition	Temperature Range (°C)	Remarks
Idle	42–48	Stable baseline temperature
Inference Only	55–62	Normal AI workload
Inference + MJPEG	60–68	Moderate increase
Inference + H264	62–70	Stable under hardware encoding
Peak Recorded	∼72	No throttling observed

8.3. Thermal Throttling and Performance Stability

The Raspberry Pi 4 is designed to initiate thermal throttling at approximately 80°C (soft throttle) and 85°C (hard throttle). During testing, the `vcgencmd get_throttled` command consistently returned `0x0`, confirming that no throttling events, frequency scaling, or undervoltage warnings occurred.

Performance remained highly stable over extended periods, as detailed below:

Table 4: Performance Stability After 30 Minutes of Load

Metric	Initial	After 30 Minutes
Infer FPS (Float16)	∼4.8	∼4.7
Infer FPS (INT8)	∼6.5	∼6.4
HLS Stream FPS	Stable	Stable
Memory Usage	Stable	Stable

8.4. Thermal Recommendations

Based on the stable thermal behavior, the system is well-suited for continuous roadside monitoring. For long-duration outdoor deployments, the following measures are recommended:

- Utilize the Float16 model to reduce CPU heat generation.
- Limit the inference FPS and stream resolution where appropriate.
- Deploy the unit within an aluminum casing providing adequate ventilation, optionally supplemented by a small 5V cooling fan for extreme environments.

9. Optimization Techniques Implemented

To achieve stable performance on limited hardware, the following optimizations were applied:

- Model quantization (Float16 and INT8)
- Reduced input resolution (320×320)
- Multi-threaded TFLite inference
- Background (non-blocking) clip saving
- Cooldown-based event triggering
- MJPEG/MP4 streaming for low-latency dashboard output

10. Dashboard and Local Deployment

The Flask-based dashboard provides:

- Real-time MJPEG/MP4 stream
- JSON-based event logging
- Downloadable event clips
- Automatic storage cleanup beyond configured limits

10.1. Example Endpoints and Storage

- Base URL: `http://127.0.0.1:5000`
- Live Stream Endpoint: `/stream` (full: `http://127.0.0.1:5000/stream`)
- Event Log Endpoint: `/events` (full: `http://127.0.0.1:5000/events`)
- Clip Storage Directory: `/home/sxc-mdts/edge-pothole-detection/detections`

10.2. Dashboard Interface and Visuals

The updated dashboard interface displays real-time statistics including Camera/Video FPS, Inference FPS, and the best detection score. It also features a sidebar for logging recent events with timestamps and confidence scores.

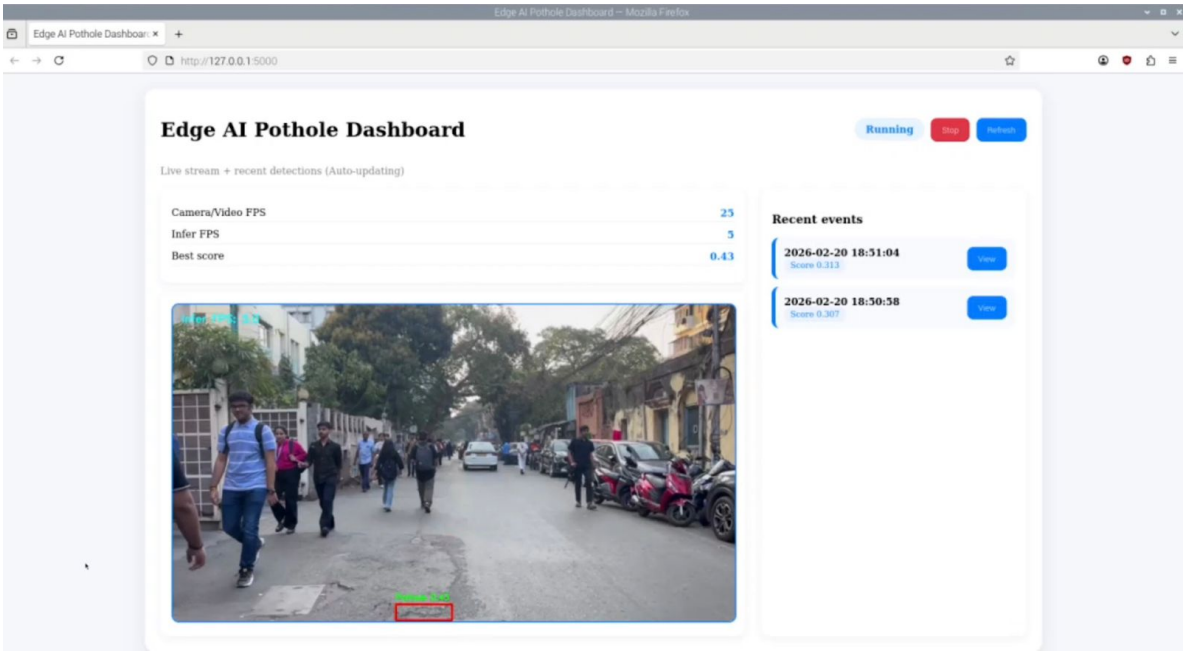

Figure 4: Live Edge AI Pothole Dashboard interface displaying real-time stream, detection overlays, inference statistics, and recent event logs.

Figure 5: Recent events logging panel from the dashboard.

11. Video and Report Analysis

An analysis was conducted on two uploaded MP4 videos to characterize video artifacts and relate them to the system's measured performance on the Raspberry Pi.

11.1. Video Technical Summary

The table below summarizes the technical specifications of the analyzed clips.

Table 5: Video Technical Summary

File	Resolution	FPS	Duration	Frames	Codec	Size
2026-02-20 18-50-31.mp4	1280x720	60.00	261.27s	15676	h264	17.1 MB
2026-02-20 21-12-34 (2).mp4	1280x720	60.00	5.50s	330	h264	249.4 KB

11.2. Motion and Freeze Characteristics

A lightweight heuristic was computed by sampling frames and measuring the mean absolute pixel difference between successive samples. Very low differences indicate static content or repeated frames that would visually feel like stutter.

Table 6: Motion / Freeze Characteristics

File	Samples	Mean diff	Freeze ratio (¡1.0)
2026-02-20 18-50-31.mp4	119	1.065	55.5%
2026-02-20 21-12-34 (2).mp4	10	0.149	100.0%

The longer video exhibits substantial static periods, which is consistent with a dashboard/screen-recording style video where only small UI elements change, or with repeated frames during buffering.

11.3. Lag and Buffering Assessment

The measured inference throughput on the Raspberry Pi is roughly 4-6 FPS for Float16 or 1-2 FPS for INT8, while the input stream operates at approximately 25-30 FPS. If the dashboard attempts to render every camera frame synchronously, the system must either drop frames or buffer them, which is perceived as lag. This mismatch between input FPS and inference FPS, combined with synchronous work, is a common root cause for excessive buffering on the Raspberry Pi. Furthermore, the original Flask/MJPEG approach can bottleneck on JPEG encoding and network throughput.

11.4. Actionable Recommendations

Based on these findings, the following optimizations are recommended to mitigate buffering:

- Decouple capture, inference, streaming, and clip-writing by using a multi-threaded architecture with a 'latest-frame' buffer. This prevents backpressure from the browser from slowing inference.
- Cap inference FPS (e.g., 5-8 FPS) and explicitly drop frames. This matches the measured Pi throughput and eliminates accumulating queues.
- Prefer H.264 (hardware) streaming for live view (HLS/WebRTC/RTSP) rather than MJPEG for sustained operation. MJPEG is easy but CPU/bandwidth heavy.
- Keep event clip saving asynchronous; if browser compatibility conversion is required, run ffmpeg in a background job, not inside the main frame loop.
- If latency is more important than visual smoothness, lower stream resolution (e.g., 480p) and reduce overlay complexity (text/rectangles) to cut CPU.

12. Conclusion

The Edge AI pothole detection system successfully demonstrates stable real-time deep learning inference on Raspberry Pi hardware.

Key findings include:

- Float16 provides optimal deployment trade-off.
- 25–35% inference speed improvement over Float32.

- Detection accuracy preserved under quantization.
- Robust performance under challenging environmental conditions.
- Stable long-duration runtime without crash.

The system is suitable for scalable roadside and vehicle-mounted edge deployment.

13. Future Scope

Future enhancements may include:

- Coral TPU acceleration
- CAN bus integration
- Edge-to-cloud hybrid analytics
- Expanded dataset training
- City-scale deployment architecture
- While the baseline system operates on 4GB of RAM, upgrading to an 8GB Raspberry Pi 4 model offers distinct architectural advantages for this Edge AI pipeline, particularly in mitigating buffering and latency issues:
  - **Expanded Frame Buffering:** The decoupled, multi-threaded pipeline can allocate significantly larger queue sizes for the 'latest-frame' buffer. This prevents backpressure from the browser or Flask server from slowing down the inference thread during sudden spikes in CPU load.
  - **In-Memory I/O Operations (tmpfs):** Extra memory permits the creation of a larger RAM disk (`tmpfs`). Background event clips and FFmpeg encoding processes can be handled entirely in volatile memory before the final MP4 is flushed to the microSD card. This vastly reduces storage I/O bottlenecks and mitigates SD card wear.
  - **Enhanced Concurrent Processing:** The 8GB capacity provides ample headroom for the OS to smoothly juggle the camera thread, AI inference, MJPEG/H.264 streaming, and the Flask dashboard simultaneously without triggering swap memory usage, which historically causes severe system stalls.
  - **Note on Inference FPS:** Because raw TFLite inference is a compute-bound task dependent on the quad-core Cortex-A72 CPU, increasing RAM to 8GB will stabilize pipeline concurrency and reduce lag, but it will not drastically increase the raw YOLOv8 inference speed (which remains at 4–6 FPS for Float16).

CERTIFICATE OF COMPLETION

This is to certify that the project titled "**Real-Time Road Anomaly Detection from Dashcam Footage on Raspberry Pi**" has been successfully carried out by **Mr. Rounak Sengupta, Mr. Saurabh Mishra, and Mr. Subhayan Biswas**, students of the Postgraduate Department of Data Science at **St. Xavier's College (Autonomous), Kolkata**, under my supervision and guidance.

This project work is submitted in partial fulfillment of the project requirements and is a bona fide record of the work carried out by the above-mentioned students during the academic session.

The work presented in this report has not been submitted elsewhere for any other academic qualification.

Signature: Aparajita Datta.

Prof. Aparajita Datta
Project Mentor & Supervisor
St. Xavier's College, Kolkata